

Indexing 2: 结构化文档、压缩与通配查询

图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

structured documents → extent index → compression motivation → delta encoding → v-byte encoding → lexicon data structures → wild-card queries → character n-gram indexes

10/1/25

Structured Documents

- Document are not always flat:
 - Meta-data: title, author, time-stamp
 - Structure: headline, section, body
 - Tags: link, hashtag, mention
- How to deal with it?
 - Neglect!
 - Create separate index for each field
 - Use "extent index"

Walid Magdy, TTDS 2025/2026



3

Extent Index

- Special "term" for each element/field/tag
 - Index all terms in a structured document as plain text
 - Terms in a given field/tag get special additional entry
 - Posting: spans of window related to a given field
 - Allows multiple overlapping spans of different types

he	→	1,1	1,5	2,1	3,3	4,3	5,1	
drink	→	1,8	2,4	2,6	2,8	3,6	4,5	5,6
ink	→	3,8	4,2	5,8				
pink	→	4,8	5,7					
Link	→	3,1:2	4,1:4	5,7:8				

D1: He likes to wink, he likes to drink
D2: He likes to drink, and drink, and drink
D3: The thing he likes to drink is ink
D4: The ink he likes to drink is pink
D5: He likes to wink, and drink pink ink

D1: He likes to wink, he likes to drink

D2: He likes to drink, and drink, and drink

D3: The thing he likes to drink is ink

D4: The ink he likes to drink is pink

D5: He likes to wink, and drink pink ink

Walid Magdy, TTDS 2025/2026



4

2

原 PPT 第 2 页

这节课一句话

这节课是在 inverted index 上“加装备”：让它懂字段、少占空间、还能处理 mon* 这种模糊词形查询。

1. 结构化文档：文档不是一块纯文本

10/1/25

Structured Documents

- Document are not always flat:
 - Meta-data: title, author, time-stamp
 - Structure: headline, section, body
 - Tags: link, hashtag, mention
- How to deal with it?
 - Neglect!
 - Create separate index for each field
 - Use "extent index"

Walid Magdy, TTDS 2025/2026

3

Extent Index

- Special "term" for each element/field/tag
 - Index all terms in a structured document as plain text
 - Terms in a given field/tag get special additional entry
 - Posting: spans of window related to a given field
 - Allows multiple overlapping spans of different types

he	→	1,1	1,5	2,1	3,3	4,3	5,1	
drink	→	1,8	2,4	2,6	2,8	3,6	4,5	5,6
ink	→	3,8	4,2	5,8				
pink	→	4,8	5,7					
Link	→	3,1:2	4,1:4	5,7:8				

D1: He likes to wink, he likes to drink

D2: He likes to drink, and drink, and drink

D3: [The thing](#) he likes to drink is ink

D4: [The ink he likes](#) to drink is pink

D5: He likes to wink, and drink [pink ink](#)

Walid Magdy, TTDS 2025/2026

4

2

原 PPT 第 2 页

人话直觉

网页像一栋楼：标题、正文、链接、hashtag 是不同房间；同一个词出现在标题里和角落脚注里，分量可能不一样。

精确定义 / 机制：structured documents 有 metadata、headline、section、body、tags 等结构。处理方式包括忽略结构、为每个 field 建独立 index，或使用 extent index 把结构 span 也纳入索引。

考试问法：若问 structured documents 如何处理，列出 neglect、separate field indexes、extent index，并说明各自直觉。

2. Extent index: 把字段也当成特殊 term

10/1/25

Using Extent

- Doc: 1 → 1 2 3
Headline: "Information retrieval lecture"
Text: "this is lecture 6 of the TTDS course on IR"
4 5 6 7 8
- Query → Headline: lecture

lecture → [1,3] [1,4] [2,9] [3,7] [3,11]

Headline → [1,1:3] [2,1:5] [3,1:4]

1 2 3 4 5 6 7 8

this is lecture 6 of the TTDS course on IR

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY OF EDINBURGH

Index Compression

- Inverted indices are big
 - Large disk space → large I/O operations
- Index compression
 - Reduce space → less I/O
 - Allow more chunks of index to be cached in memory
- Large size goes to:
 - terms? document numbers?
 - Ideas:
 - Compress document numbers, how?

 THE UNIVERSITY
of EDINBURGH

Walid Magdy, TTDS 2025/2026

6

3

原 PPT 第 3 页

人话直觉

extent 像给文本贴彩色胶带: headline 从 1 到 3, body 从 4 到 8; 查询时看词有没有落在胶带范围里。

精确定义 / 机制： extent index 为每个 element/field/tag 建特殊 entry，同时普通 terms 仍按 plain text 索引。posting 可记录某字段在文档中的 span，例如 Headline 1,1:3。这样可以支持 Headline:lecture 这类字段约束查询，也允许不同类型 span overlap。

考试问法：考试答 extent index 时，关键词是 special term、field/tag spans、overlapping spans、支持 fielded search。

3. 为什么要压缩 index

10/1/25

Index Compression

- There are more sophisticated compression algorithms:
 - Elias gamma code
- The more compression
 - Less storage
 - More processing
- In general
 - Less I/O + more processing > more I/O + no processing
" > " = faster
 - With new data structures, problem is less severe

Walid Magdy, TTDS 2025/2026

9

Dictionary Data Structures

- The dictionary data structure stores the term vocabulary, document frequency, pointers to each postings list ...
- For small collections, load full dictionary in memory. In real-life, cannot load all index to memory!
 - Then what to load?
 - How to reach quickly?
 - What data structure to use for inverted index?

Walid Magdy, TTDS 2025/2026

10

5

原 PPT 第 5 页

人话直觉

索引太大像背着砖头跑步：不只是占硬盘，还会让每次从磁盘搬数据都变慢。

精确定义 / 机制：inverted indices 很大，存储空间和 I/O 都昂贵。压缩能减少 disk space、降低 I/O，并让更多 index chunks 缓存在内存。代价是查询时需要解码，形成 less I/O + more processing 与 more I/O + no processing 的权衡。

考试问法：若问 index compression 好处，不要只说省空间；一定加上减少 I/O 和提升 cache 命中。

4. Delta encoding: 存 gap 而不是存完整 docID

10/1/25

Delta Encoding

- Large collections → large sequence of doc IDs
 - e.g. Doc IDs: 1, 2, 3, ... 66,032, ..., 5,323,424,235
- Large ID number → more bytes to store
 - 1 byte: 0 → 255
 - 2 bytes: 0 → 65,535
 - 4 bytes: 0 → 4.3 B
- Idea: delta in ID instead of full ID
 - Very useful, especially for frequent terms

term → ... 100002 100007 100008 100011 100019 ...

term → ... 7 5 1 3 7 ... 321 15 2 ...

1 byte 2 bytes 3 bytes

Wahid Magdy, TTDS 2025/2026 THE UNIVERSITY OF EDINBURGH

v-byte Encoding

- Have different byte storage for each delta in index
 - Use fewer bits to encode
 - High bit in a byte → 1/0 = terminate/continue
 - Remaining 7 bits → binary number
 - Examples:
 - "6" → 10000110
 - "127" → 11111111
 - "128" → 00000010000000
- Real example sequence:
 - 10001010 00000001 10000010 0000111
 - 0000101 → 000000010000010 → 0000111
 - 5 → 130 → 7

Wahid Magdy, TTDS 2025/2026 THE UNIVERSITY OF EDINBURGH

4

原 PPT 第 4 页

人话直觉

如果朋友住 100002、100007、100008 号，记“100002 后面 +5、+1”比每次写完整门牌轻多了。

精确定义 / 机制：postings list 中 docIDs 递增排列。对大 collection，完整 docID 可能很大，需要多字节存储；delta encoding 存相邻 docID 的差值。频繁 term 的 postings list 通常 gap 较小，因此特别适合压缩。

$$gap_i = docID_i - docID_{i-1}$$

考试问法：若给 docID 序列让你压缩，第一项常保留或作为起点，后续写相邻差值。

5. v-byte encoding: 小 gap 少用字节

10/1/25

Delta Encoding

- Large collections → large sequence of doc IDs
 - e.g. Doc IDs: 1, 2, 3, ... 66,032, ..., 5,323,424,235
- Large ID number → more bytes to store
 - 1 byte: 0 → 255
 - 2 bytes: 0 → 65,535
 - 4 bytes: 0 → 4.3 B
- Idea: delta in ID instead of full ID
 - Very useful, especially for frequent terms

term → ... 100002 100007 100008 100011 100019 ...

term → ... 7 5 1 3 7 ... 321 15 2 ...

1 byte 2 bytes 3 bytes

Walid Magdy, TTDS 2025/2026 THE UNIVERSITY OF EDINBURGH

v-byte Encoding

- Have different byte storage for each delta in index
 - Use fewer bits to encode
 - High bit in a byte → 1/0 = terminate/continue
 - Remaining 7 bits → binary number
 - Examples:
 - "6" → 10000110
 - "127" → 11111111
 - "128" → 00000010000000
- Real example sequence:
 - 10001010 00000001 10000010 10000111
 - 0000101 → 0000000100000010 → 0000111
 - 5 → 130 → 7

Walid Magdy, TTDS 2025/2026 THE UNIVERSITY OF EDINBURGH

4

原 PPT 第 4 页

人话直觉

不是每个数字都发一个大箱子；小数字用小信封，大数字才用多几个信封。

精确定义 / 机制： variable-byte encoding 为每个 delta 使用不同数量的 byte。每个 byte 中高位作为 continue/terminate 标志，其余 7 bits 存数值。它比固定长度整数更节省空间，同时解码比极端 bit-level 编码简单。

考试问法： 考试通常考概念：v-byte 用 continuation bit + 7-bit payload，以变长方式存 gap。

6. 压缩不是越狠越无脑

10/1/25

Index Compression

- There are more sophisticated compression algorithms:
 - Elias gamma code
- The more compression
 - Less storage
 - More processing
- In general
 - Less I/O + more processing > more I/O + no processing
" > " = faster
 - With new data structures, problem is less severe

Walid Magdy, TTDS 2025/2026

9

Dictionary Data Structures

- The dictionary data structure stores the term vocabulary, document frequency, pointers to each postings list ...
- For small collections, load full dictionary in memory. In real-life, cannot load all index to memory!
 - Then what to load?
 - How to reach quickly?
 - What data structure to use for inverted index?

Walid Magdy, TTDS 2025/2026

10

5

原 PPT 第 5 页

人话直觉

把衣服压缩进真空袋很省空间，但每次取衣服要放气；压缩算法也是这样。

精确定义 / 机制：更复杂的压缩如 Elias gamma code 可进一步减少空间，但会增加编码/解码成本。IR 中常见判断是：如果减少 I/O 带来的收益大于额外 CPU 解码成本，压缩就是划算的。

考试问法：若问 trade-off，答 storage/I/O 降低 vs processing 增加，并说明实际常常 I/O 是瓶颈。

7. Lexicon data structures: hash 快, tree 顺

10/1/25

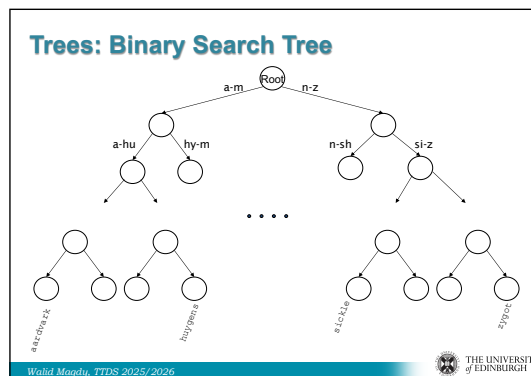
Hashes

- Each vocabulary term is hashed to an integer
- Pros
 - Lookup is faster than for a tree: $O(1)$
- Cons
 - No easy way to find minor variants:
 - judgment/judgement
 - No prefix search
 - If vocabulary keeps growing, need to occasionally do the expensive operation of rehashing everything

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

11



12

6

原 PPT 第 6 页

人话直觉

hash 像知道门牌直接瞬移；B-tree 像按字母排好的电话簿，虽然慢一点，但能顺着找 mon 开头的一串名字。

精确定义 / 机制：lexicon 存 vocabulary terms 及 postings pointers。hash lookup 平均 $O(1)$ ，但不支持前缀、范围或通配遍历；tree/B-tree lookup 为 $O(\log M)$ ，但保留字典序，适合 prefix search、range traversal 和磁盘友好访问。

$$\text{hash} : O(1), \quad \text{B-tree} : O(\log M)$$

考试问法：若问 hash vs B-tree，不要只背复杂度；要联系 wildcard/prefix query 的需求。

8. Wildcard queries: mon* 容易, *mon 麻烦

10/1/25

Wild-Card Queries: *

- mon*: find all docs containing any word beginning "mon".
- Easy with binary tree (or B-tree) lexicon
- *mon: find words ending in "mon": harder
 - Maintain an additional B-tree for terms backwards.
- How can we enumerate all terms meeting the wild-card query pro*cent ?
- Query processing: se*ate AND fil*er ?
 - Expensive

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

15

Permuterm Indexes

- Transform wild-card queries so that the * occurs at the end
- For term *hello*, index under:
 - *hello\$, ello\$h, llo\$he, lo\$hek, o\$shell, \$hello*
where \$ is a special symbol.
- Rotate query wild-card to the end
- Queries:
 - X lookup on X\$
 - X* lookup on \$X*
 - *X lookup on
 - X*Y lookup on
- Index Size?

Example

hello	→	hello\$	→	\$hel*
hel*	→	hel*\$	→	\$hel*
*llo	→	*llo\$	→	llo\$*
he*lo	→	he*lo\$	→	lo\$he*

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

16

8

原 PPT 第 8 页

人话直觉

按姓氏找人容易，因为电话簿按姓排；按名字最后一个字找人，就得换一本反向电话簿。

精确定义 / 机制：mon* 可在有序 lexicon 中定位 mon 前缀区间；*mon 需要 reverse lexicon 才方便；m*n 这类中间通配更复杂，通常需把候选 terms 先找出来再过滤。wild-card search 的核心不是直接查文档，而是先在 vocabulary 中展开匹配的 terms。

考试问法：若问 wildcard processing，要说两阶段：在 lexicon 找匹配 terms，然后把这些 terms 的 postings 合并。

9. Character n-gram index: 给词项拆零件

10/1/25

Character n-gram Indexes: Query time

- Step 1: Query **mon*** → **\$m AND mo AND on**
 - It would still match **moon**.
- Step 2: Must post-filter these terms against query.
 - Phrase match, or post-step1 match
- Step 3: Surviving enumerated terms are then looked up in the term-document inverted index.
→ **Montreal OR monster OR monkey**
- Wild-cards can result in expensive query execution (very large disjunctions...)

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

19

Character n-gram Indexes: Applications

- Spelling Correction
 - Create n-gram representation for words
 - Build index for words:
 - Dictionary of words → documents (each word is a document)
 - Character n-grams → terms
 - When getting a search term that is misspelled (OOV or not frequent), find possible corrections
 - Possible corrections = most matching results

Query: **elephant** → **\$e el le ep pg ga an nt t\$**

Results:

elegant → **\$e el le eg ga an nt t\$**

elephant → **\$e el le ep ph ha an nt t\$**

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

20

10

原 PPT 第 10 页

人话直觉

把 month 拆成 mo、on、nt、th；查询 mon* 时先找含这些零件的候选词，再验明正身。

精确定义 / 机制：character n-gram index 枚举 vocabulary terms 中出现的字符 n-grams。query time 把 wildcard pattern 转成 n-gram 约束，例如 mon* 可产生 \$m、mo、on 等约束，先求候选 terms，再用原 pattern post-filter，避免 moon 等误匹配。char n-grams 还可用于 OCR 错误、拼写变体或无成熟 stemmer 的语言。

考试问法：常考 n-gram index 的 false positives：第一步只产生候选，必须 post-filter。

考试速记版

- Structured documents 可忽略结构、分 field 建 index，或用 extent index。
- Extent index 把 field/tag span 作为特殊 term，支持字段约束查询。

- Index compression 主要收益是省空间、少 I/O、更多缓存；代价是解码 CPU。
- Delta encoding 存相邻 docID 的 gap，频繁 term 尤其受益。
- v-byte 用变长 byte 表示 gap，高位做继续/终止标记。
- Wildcard query 先在 lexicon 展开匹配 terms，再合并 postings。
- Character n-gram index 会产生候选，需要 post-filter。

一段稳妥考场回答

Indexing 2 扩展了基本 inverted index。对于 structured documents，可以忽略结构、为字段分别建索引，或用 extent index 把 headline、body、tag 等字段 span 作为特殊 term 存入索引，从而支持 fielded search。由于 inverted index 很大，compression 通过 delta encoding 和 v-byte 等方法减少存储与 I/O，让更多数据进入 cache，但会增加解码处理。对于 wildcard queries，系统通常先在 lexicon 中找出匹配词项，再合并 postings；prefix 查询适合 B-tree，复杂 wildcard 可借助 character n-gram index 生成候选并 post-filter。